SYSTEMATIC EVALUATION AND EXTENSION OF DEPENDENCY GRAPH-BASED STRUCTURAL PRUNING ACROSS CONVOLUTIONAL, GRAPH AND TRANSFORMER NEURAL NETWORKS

CO3554 DATA MANAGEMENT PROJECT

PROJECT PROPOSAL

NAME : PATHIRAGE L.P.D.P.G

INDEX NO : 23/ENG/091

REGISTRATION NO : EN113719

SUBMISSION DATE : 21/10/2026

TABLE OF CONTENTS

1 Introduction **2**

2 Background **3**

3 Aims and Objectives **5**

4 Implementation Plan **6**

5 Milestones and Timeline **10**

6 References **12**

1. INTRODUCTION

Deep neural networks now power image recognition, language understanding and graph analysis. Their accuracy, however, comes from very large numbers of parameters. A standard ResNet-50 needs about 4.1 billion multiply accumulate operations (MACs) for one image, and a ViT-B/16 Transformer needs about 17.6 billion [1]. Running such models on edge devices such as phones, drones or embedded boards like the NVIDIA Jetson is difficult because of limited memory, power and compute.

Pruning is one of the most practical ways to reduce this cost. The idea is simple: many parameters in a trained network are not important, so they can be removed with little loss in accuracy [3]. There are two main types:

- **Unstructured pruning** sets individual weights to zero. The model keeps the same shape, so real speed-up needs special sparse hardware or software, or file compression [11].
- **Structural pruning** removes whole channels, neurons or attention heads. The model physically becomes smaller and runs faster on ordinary hardware [4].

Structural pruning has a hidden difficulty: layers are *coupled*. If one output channel of a convolution is removed, the matching channel in the next batch-normalisation layer and the matching input channel of the next convolution must also be removed, otherwise the network breaks (Figure 1). In modern networks with residual connections, concatenations and attention, these links become very complex.

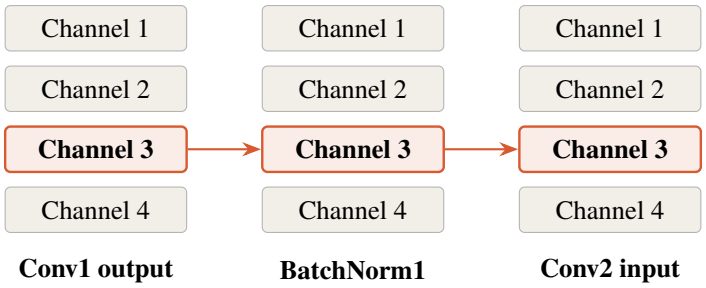


Figure 1: Coupled parameters. Removing channel 3 from Conv1 forces the removal of channel 3 in BatchNorm1 and of input channel 3 in Conv2. These linked parts form one *group* that must be pruned together.

Fang et al. [1] proposed the **Dependency Graph (DepGraph)**, which models these links automatically and groups the coupled parameters of any network. It is implemented in the open-source **Torch-Pruning** library [2], which is widely used and has been extended to large language models [9]. The paper claims the method supports *any* architecture, including CNNs, RNNs, GNNs and Transformers.

This claim, however, has limited evidence outside vision. In the original paper, the non-image experiments use one LSTM, one DGCNN and one GAT model (Table 4 of [1]). There is no independent, systematic study of which layers and operators the library really handles, and it is not known how often pruning fails silently. This project addresses that gap.

2. BACKGROUND

2.1 Neural network cost

A neural network is a chain of layers (convolutions, linear layers, normalisation, attention, recurrent cells). Each layer stores weights and performs multiplications. The main cost measures used in this project are:

- **Parameters** - the number of weights (affects model size and memory);
- **MACs** - multiply - accumulate operations per input (theoretical compute);
- **Latency** - the real time to process one input on a given device.

MACs and latency do not always agree, so measuring real latency is important.

2.2 Pruning pipeline and importance criteria

A typical structural pruning pipeline has three steps: (1) start from a trained model, (2) score each prunable unit and remove the least important ones, (3) fine-tune the smaller model to recover accuracy. Common scoring (importance) criteria are:

- **Magnitude (L1/L2 norm)** - small weights are assumed to matter less [4];
- **Batch-norm scaling factors** - used in Network Slimming [5];
- **Taylor / gradient-based** - estimates how much the loss changes if a unit is removed [6];
- **Random** - a simple baseline that is surprisingly hard to beat in some settings.

2.3 The dependency problem and DepGraph

DepGraph splits every layer into an input part and an output part, and adds an edge between two parts when they must be pruned together. Two rules create the edges [1]:

- **Inter-layer dependency**: connected layers share the same feature map, so the output of one layer and the input of the next are linked.
- **Intra-layer dependency**: some layers (e.g. batch norm, element-wise operations) must prune their input and output in the same way.

Searching this graph returns every group of coupled parameters. DepGraph then scores whole groups and uses sparse training so that all members of an unimportant group become small together. With a simple norm criterion it reported results comparable to more complex methods on CIFAR and ImageNet.

2.4 The Torch-Pruning library

Torch-Pruning traces a PyTorch model, builds the dependency graph and offers pruners with different importance criteria. Support for each operator type (convolution, linear, normalisation, multi-head attention, LSTM, etc.) is implemented as a separate "pruner" function inside the library. Any operator without such a function, or with an incorrect one, can lead to a crash or to a wrong model.

2.5 Gaps in current knowledge

1. **Limited non-vision testing.** Only one model per non-vision family was evaluated in the original paper.
2. **No support map.** There is no public, systematic record of which layers and model families work, crash, or produce silent errors.

3. **Under-served layers.** Recurrent layers such as stacked LSTMs and GRUs, and graph layers from PyTorch Geometric [10], have limited or untested support in the library.
4. **Unfair comparisons.** Pruning papers often use different code, training budgets and baselines, which makes results hard to compare [7, 8]. Same-code comparisons on non-vision architectures are rare.

3. AIMS AND OBJECTIVES

This project aims to independently and systematically measure how well dependency graph-based structural pruning works across CNN, Transformer, RNN and GNN architectures, to extend it where it fails, and to make the results easy to understand through a fair benchmark and an interactive visualisation dashboard. The following are the other aims and objectives of this project.

- Build an automated test harness that applies Torch-Pruning to at least 30 well-known models from four families (CNN, Transformer, RNN, GNN) and records each result as *success*, *crash* or *silent error*.
- Design a correctness check that detects silent errors automatically, by comparing each pruned model with an equivalent masked model.
- Produce a support map showing which layers, operators and model families the library handles correctly, and find the root causes of the failures.
- Extend the library with pruning support for at least one under-served layer type (e.g. GRU, stacked LSTM or a PyTorch Geometric layer), with tests, and submit it to the open-source project.
- Fairly compare pruning methods (same code, same training budget) on at least two non-vision architectures, using accuracy, parameters, MACs and measured latency.
- Develop an interactive dashboard that reads all experiment results from a results database and visualises the support map, the channels and layers removed in each dependency group, and the trade-off between pruning ratio, accuracy, model size and latency.

4. IMPLEMENTATION PLAN

4.1 Datasets

The project uses the same public benchmark datasets as the DepGraph paper [1], so that results can be compared directly with the original work, plus one small graph dataset (Cora) that is quick to train on a laptop. All datasets are freely available and can be downloaded through standard libraries (torchvision, Hugging Face Datasets and PyTorch Geometric). Table 1 summarises them.

Dataset	Data type	Size	Classes	Used in
CIFAR-10 [12]	Images (32×32)	50,000 train / 10,000 test	10	Stages 1, 4
CIFAR-100 [12]	Images (32×32)	50,000 train / 10,000 test	100	Stage 4
ImageNet-1k [13]	Images (high-res.)	1.28M train / 50,000 val	1,000	Stage 4 (val only)
AG News [14]	Text (news articles)	120,000 train / 7,600 test	4	Stage 4
PPI [15]	Graphs (proteins)	24 graphs, $\sim$57,000 nodes	121 labels	Stage 4
Cora [16]	Graph (citations)	2,708 nodes, 5,429 edges	7	Stages 3, 4
ModelNet40 [17]	3D point clouds	9,843 train / 2,468 test	40	Stretch goal

Table 1: Datasets used in the project. Datasets marked "Stage 4" are used for the fair benchmark; the test harness (Stage 2) does not need a dataset.

How each dataset is used.

- **Test harness (Stage 2).** Checking whether pruning *works* does not need real data. Each model is fed random input tensors of the correct shape (e.g. $1 \times 3 \times 224 \times 224$ for an image model, or a random token sequence for a text model), and the pruned and masked outputs are compared. This keeps the harness fast and independent of downloads.
- **CIFAR-10 and CIFAR-100.** Small image datasets used to reproduce the DepGraph CNN results (Stage 1) and to measure how accuracy changes after pruning and fine-tuning. They fit easily on a laptop or Colab.
- **ImageNet-1k.** Too large to train on with the available hardware (about 150 GB). Instead, *pretrained* torchvision models are pruned and evaluated only on the 50,000-image validation set (about 6.4 GB), as in the DepGraph paper. If storage is a problem, a fixed random subset of the validation set will be used and reported clearly.
- **AG News.** Text classification dataset for the RNN (LSTM/GRU) and Transformer (BERT, DistilBERT) experiments.
- **PPI and Cora.** Graph datasets for the GNN experiments. PPI matches the DepGraph paper; Cora is small and fast, so it is used first while developing the GNN pruning extension (Stage 3).
- **ModelNet40.** 3D point-cloud dataset used by DepGraph for its DGCNN result; included only as a stretch goal.

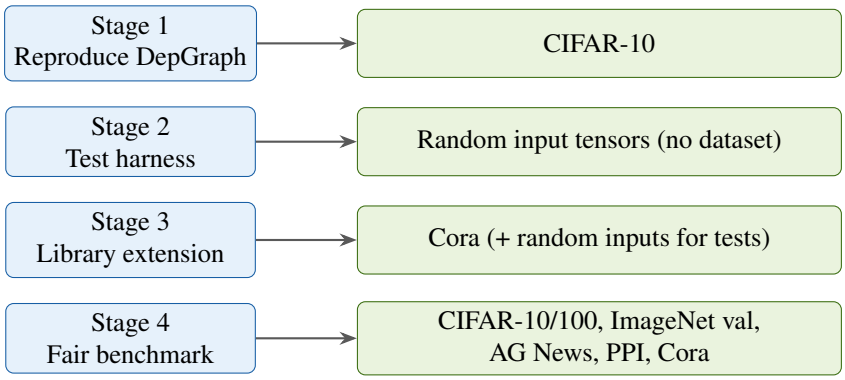


Figure 2: Which data each stage of the project uses.

4.2 Implementation process

The work is carried out in four stages that build on each other: (1) learning and reproduction, (2) the test harness, (3) library extension, and (4) benchmarking. All results will be stored in a results database and shown through a simple dashboard. Figure 3 shows the core harness.

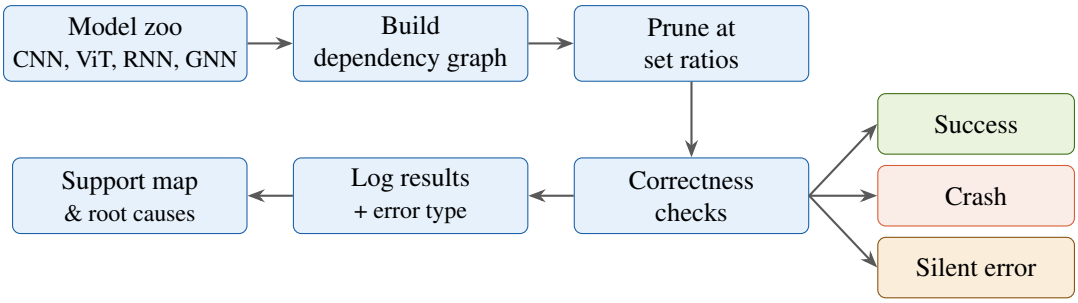


Figure 3: The automated test harness. Each model is pruned at several ratios, checked, and classified into one of three outcomes.

4.2.1 Stage 1 - Learning and reproduction

Study PyTorch, CNN training and the core pruning papers. Reproduce a small result from [1] (e.g. pruning ResNet-56 or ResNet-18 on CIFAR-10) using Torch-Pruning, to confirm the setup works and to learn the library's code structure.

4.2.2 Stage 2 - Test harness

Model zoo. Pretrained or randomly initialised models are loaded from standard sources (Table 2). Random weights are enough to test whether pruning *works*; trained weights are needed only for the benchmark.

Family	Example models	Source
CNN	ResNet-18/50, MobileNetV2/V3, DenseNet-121, EfficientNet-B0, ConvNeXt-T, RegNet	torchvision
Transformer	ViT-B/16, Swin-T, BERT-base, DistilBERT, GPT-2 small	torchvision, Hugging Face
RNN	LSTM (single, stacked, bidirectional), GRU (single, stacked)	PyTorch `nn`
GNN	GCN, GAT, GraphSAGE, GIN	PyTorch Geometric

Table 2: Planned model zoo (final list may change; target ≥ 30 models).

Outcome classes. For each model and pruning ratio the harness records:

- **Success** - pruning finishes, the model runs, and the correctness check passes;
- **Crash** - an exception occurs while building the graph, pruning, or running the pruned model (error type and layer are logged);
- **Silent error** - the model runs, but the correctness check fails.

Correctness check (silent-error detection). A pruned model is smaller, so its outputs cannot simply be compared with the original. Instead, a *masked* copy of the original model is created in which every parameter slice selected for removal (in all members of each group) is set to zero. Physically removing those slices should give exactly the same outputs as the masked model. The harness therefore checks:

1. the pruned model runs a forward pass with the expected output shape;
2. the outputs of the pruned and masked models match within a small numerical tolerance;
3. the parameter and MAC counts drop as expected.

The exact design of this check will be refined with the supervisors.

Root-cause analysis. Failures are grouped by the operator or pattern that caused them (e.g. recurrent weights, attention head reshaping, message passing in GNNs), producing the support map.

4.2.3 Stage 3 – Library extension

Based on the support map, one or two of the most important gaps will be selected (likely candidates: GRU, stacked LSTM, a PyTorch Geometric layer such as `GCNConv` or `GATConv`). For each, the project will: study how its weights are coupled; implement a pruner function in Torch-Pruning; write unit tests using the harness's correctness check; and open a pull request to the public repository.

4.2.4 Stage 4 – Fair benchmark

At least two non-vision architectures (e.g. an LSTM/GRU text classifier on AG News and a GAT/GCN model on Cora or PPI; see Section 4.1) will be pruned with several criteria inside the same code base and with the same fine-tuning budget:

- Random, L1 magnitude, Taylor, BN-scaling (where applicable), and DepGraph group-norm importance.

Each setting is run with several random seeds. Reported metrics: accuracy (or F1), parameters, MACs, and measured latency on a laptop CPU/GPU, and on a Jetson Orin Nano if one is available.

4.3 Technology stack

The following tools and technologies will be used:

- Programming language : Python
- Deep learning framework : PyTorch
- Pruning library : Torch-Pruning
- Model libraries : torchvision, Hugging Face Transformers, PyTorch Geometric
- Development environment : VS Code, Jupyter Notebook, Google Colab
- Version control : Git, GitHub

4.4 Expected outcomes and evaluation

- An open-source test harness and a public support map of Torch-Pruning across four model families (evaluated by number of models and operators covered).
- New, tested pruning support for at least one under-served layer type (evaluated by passing the correctness check and by a submitted pull request).
- A fair benchmark of pruning criteria on non-vision models (evaluated by reproducibility across seeds).
- An interactive dashboard for exploring pruning results (evaluated by covering all experiments in the results database and by supervisor feedback).
- A research paper describing the methods and results, written in the ICML format (up to 8 pages, plus references and appendices). This is the written deliverable of the module.

5. MILESTONES AND TIMELINE

The project runs for 35 weeks.

The work is divided into eight tasks that are done one after another, without overlap (Figure 4). The time given to each task depends on how difficult it is. The test harness and the library extension get the most time (7 weeks each), because they involve the most new code. The benchmark experiments get 5 weeks because they need many training runs with several random seeds. Simpler tasks, such as reproducing a known result or building the dashboard, get 2 weeks.

5.1 Milestones

ID	Target week	Milestone / deliverable
M1	Week 0	Project proposal submitted
M2	Week 6	Background study done; DepGraph result reproduced on CIFAR-10
M3	Week 13	Test harness complete for ≥ 30 models
M4	Week 17	Support map and failure analysis complete; progress review
M5	Week 24	Library extension implemented, tested, pull request opened
M6	Week 31	Benchmark experiments complete; pruning dashboard working
M7	Week 35	Research paper submitted; code released

Table 3: Project milestones.

5.2 Timeline

The Gantt chart below shows each task week by week, how many weeks it takes, and when the milestones (M2-M7) are reached.

Gantt chart – Structural Pruning Project (35 weeks)

Task	Weeks	Week number (1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35)
Literature review & PyTorch basics	4	
Reproduce DepGraph (CIFAR-10)	2	
Build test harness	7	
Support map & failure analysis	4	
Library extension & tests	7	
Fair benchmark experiments	5	
Pruning dashboard	2	
Research paper writing & submission	4	

Milestones — M2 · M3 · M4 · M5 · M6 · M7

Learning · Development · Analysis · Experiments · Dashboard · Writing

Figure 4: Project Gantt chart. Each coloured square is one week. Red diamonds mark the milestones in Table 3.

6. REFERENCES

[1] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, "DepGraph: Towards any structural pruning," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 16091–16101.

[2] G. Fang, "Torch-Pruning: Towards any structural pruning," GitHub repository, 2023. [Online]. Available: https://github.com/VainF/Torch-Pruning

[3] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[4] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," in *Int. Conf. Learning Representations (ICLR)*, 2017.

[5] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, 2017, pp. 2736–2744.

[6] P. Molchanov, A. Mallya, S. Tyree, I. Frosio, and J. Kautz, "Importance estimation for neural network pruning," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2019, pp. 11264–11272.

[7] D. Blalock, J. J. Gonzalez Ortiz, J. Frankle, and J. Guttag, "What is the state of neural network pruning?" in *Proc. Machine Learning and Systems (MLSys)*, 2020.

[8] H. Wang, C. Qin, Y. Bai, and Y. Fu, "Why is the state of neural network pruning so confusing? On the fairness, comparison setup, and trainability in network pruning," *arXiv preprint arXiv:2301.05219*, 2023.

[9] X. Ma, G. Fang, and X. Wang, "LLM-Pruner: On the structural pruning of large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[11] A. Lindamulage, N. Kodagoda, S. Reyal, P. Samarasinghe, and P. Yogarajah, "Comparative study of parameter selection for enhanced edge inference for a multi-output regression model for head pose estimation," *arXiv preprint arXiv:2302.00592*, 2022.

[12] A. Krizhevsky, "Learning multiple layers of features from tiny images," Technical report, University of Toronto, 2009.

[13] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[14] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[15] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[16] P. Sen, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, and T. Eliassi-Rad, "Collective classification in network data," *AI Magazine*, vol. 29, no. 3, pp. 93–106, 2008.

[17] Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao, "3D ShapeNets: A deep representation for volumetric shapes," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 1912–1920.